

# TRABAJO PRÁCTICO INTEGRADOR 2026

## Sistema de Semáforos Inteligentes

Paradigmas y Lenguajes de Programación  
Facultad de Ciencias Exactas y Naturales y Agrimensura  
Universidad Nacional del Nordeste

---

### INTEGRANTES DEL GRUPO N° 2

Caraballo, Ivan Jeremias  
Castillo, Gonzalo Uriel  
Escalante, Enzo Leonel  
Gómez, Jonathan Daniel  
Palacios, Evangelina Victoria

---

Lenguajes utilizados: Common Lisp y Clojure

## **Introducción:**

En el siguiente informe se redactará como fue hecho el proyecto integrador del Grupo Número 2. En él se explicará cómo fue la realización de cada una de las 3 fases cumpliendo con los objetivos del trabajo elaborando soluciones a través de los lenguajes Lisp y Clojure. Además de en cada fase mencionar los inconvenientes que tuvimos para cada una de ellas.

### **Objetivos del Trabajo Práctico**

- **Modelar un problema del mundo real:** Traducir los requisitos funcionales de un sistema de tráfico urbano a código LISP utilizando el paradigma funcional.
- **Desarrollar autonomía y pensamiento algorítmico:** Resolver la integración de herramientas externas y el aprendizaje de una tecnología nueva con el mínimo de asistencia docente.
- **Análisis Crítico y Metacognición:** Evaluar y tipificar las funciones creadas, comprendiendo sus implicancias en la memoria y el flujo de datos, y comparar cómo diferentes lenguajes abordan un mismo problema lógico.

## **Fase 1: El problema de los 3 focos**

La fase 1, es el eje en donde se basa el trabajo en su totalidad, dando pie a las demás fases.

El contexto es el siguiente:

***“Las ciudades modernas requieren sistemas de tráfico inteligentes para optimizar el flujo vehicular y garantizar la seguridad vial. Su equipo ha sido contratado para desarrollar el núcleo lógico de un sistema embebido que controlará semáforos en intersecciones críticas de la ciudad. El mismo fue implementado en Common Lisp”***

Fuente: Documento [“Ejercicio Semaforos v2.1.2”](#) Cátedra de Paradigmas y Lenguajes

Bajo este contexto se cumplan los objetivos de: Modelar un problema del mundo real, Aplicar los conceptos fundamentales, Desarrollar pensamiento algorítmico y la exposición a circunstancias nuevas y desconocidas.

Y desarrollandolo en Common Lisp, en Lenguaje de Programación Lisp

El lenguaje LISP fue uno de los tantos pioneros en aplicar a la programación funcional en la programación declarativa, se la considera como un lenguaje multiparadigma ya que es orientado a objetos, reflexivo, imperativo y funcional. La palabra LISP es un acrónimo de LISt Processing.

Es un lenguaje de 5ta generación en donde se le da más protagonismo en su desempeño al software (es decir trabaja más el software) por último se caracteriza por ser un lenguaje funcional híbrido.

Para la elaboración de los requerimientos solicitados, los integrantes del grupo trabajaron en equipo para llegar a un resultado óptimo según las especificaciones técnicas.

## Requerimiento 1: Estados de Transición

El siguiente requerimiento tiene como objetivo modelar las transiciones de colores de los semáforos en una función de nombre “transición”.

Las especificaciones técnicas son las siguientes:

**Entrada:** “color-actual” (que puede ser una de las siguientes: en-rojo, en-amarillo, en-verde) y cambiar-a (que puede ser de color destino: rojo, amarillo, verde)

**Salida:** Devuelve una lista con el estado actual y la acción a realizar, que debe verse como el literal “cambiar-a-<color>”.

**Comportamiento:** Se llama a la función pasando como parámetro el color actual y color a cambiar, devolviendo una lista con el color actual y la salida ya dicha.

Si fuera inválida la transición entonces devolverá por defecto el color actual y ‘acción-por-defecto’.

El siguiente es un bloque del código realizado para hacer el requerimiento 1:

```
8 (defun transicion (color-actual cambiar-a)
9   (cond
10    ((and (eq color-actual 'en-rojo) (eq cambiar-a 'verde))
11      (list color-actual 'Cambiar-a-verde)
12    )
13    ((and (eq color-actual 'en-verde) (eq cambiar-a 'amarillo))
14      (list color-actual 'Cambiar-a-amarillo)
15    )
16    ((and (eq color-actual 'en-amarillo) (eq cambiar-a 'rojo))
17      (list color-actual 'Cambiar-a-rojo)
18    )
19    (t
20      (list color-actual 'Accion-por-defecto)
21    )
22  )
23 )
```

Figura 1. Código correspondiente del Requerimiento 1 en Lisp.

Fuente: Elaboración Propia

Como se muestra en la Figura 1, se tomó la decisión de hacer la función condicional, además de, para mantener la robustez del código, se decidió que las entradas sean exactas.

Y en caso de ser una acción por defecto, se toma en cuenta que falla el parámetro de “cambiar-a”, ya que lógicamente el parámetro de “color-actual” del semáforo ingresa sin inconvenientes.

## **Requerimiento 2: Temporizador Automático**

Para el siguiente requerimiento, la funcionalidad de este es la automatización de temporización, es decir que el temporizador de lo que dura cada color se haga de manera automática.

Para ello se implementará una función de nombre “timer” que recibe un tiempo en formato Unix, o tiempo epoch también llamado. Y de salida el color correspondiente en ese momento.

El tiempo unix/epoch es un número entero con signo que representa la cantidad de segundos transcurridos desde el 1 de enero de 1970 a las 00:00:00 horas.

Es muy usado en informática ya que permite representar horas y fechas con un número entero.

Las reglas de temporización dadas son las siguientes:

- Rojo con duración de 90 segundos
- Amarillo con duración de 6 segundos
- Verde con duración de 120 segundos

### **Especificaciones técnicas:**

**Entrada:** La entrada debe ser el tiempo unix en número entero.

**Salida:** La salida debe ser el color que corresponde en el momento dado.

**Comportamiento:** El comportamiento del mismo es devolver un color para el momento dado.

```
27 ;;Requerimiento 2
28
29 ;; =====
30 ;; FUNCION: timer
31 ;; NATURALEZA: Pura (no devuelve nada en pantalla)
32 ;; ESTRATEGIA: Seleccion condicional
33 ;; IMPACTO: No destructiva
34 ;; =====
35 (defun timer (unix-time) ;;unix-time es el tiempo epoch
36   (let ((resto (mod unix-time 216))) ;;coloque un let para evitar la repeticion
37     (cond
38       ((< resto 90) 'rojo)
39       ((< resto 210) 'verde)
40       (t
41        'amarillo)
42     )
43   )
44 )
45 )
```

Figura 2. Código correspondiente del Requerimiento 2.

Fuente. Elaboración Propia

Como se muestra en la figura 2, se recibe el tiempo en formato unix y a partir del resto obtenido al dividirlo por el tiempo total del ciclo, entrara en uno de los rangos posibles y así asignando un color correspondiente.

Si está dentro de 0 a 89 segundos, se le asignará el color rojo, si esta dentro del rango de los 90 a 209 segundos, se le asigna el verde, y si no esta en ninguno de ellos, por descarte está en el rango de entre los 210 a 215 segundos, asignando el color amarillo.

Se da por hecho que el tiempo en formato unix se pasa sin errores ya que el requerimiento define explícitamente que la entrada corresponde a un tiempo Unix entero, además de ser en tiempo actuales es un entero positivo. Además de que el objetivo de la función es implementar la lógica de temporización y automatización de los estados de los semáforos.

## Requerimiento 3: Sistema de Auditoría.

El equipo de analistas forenses necesita determinar qué color tenía un semáforo en una hora específica por ello se crea una función que mostrará en terminal los cambios de estado del semáforo.

### Especificaciones Técnicas:

#### Formato de salida:

*“Tiempo <Unix>: la luz ha cambiado de <color-anterior> a <color-nuevo>”*

**Propósito:** Permitir reconstrucción histórica de estados del semáforo

```
50 ;;Requerimiento 3
51
52 ;; =====
53 ;; FUNCION: auditoria
54 ;; NATURALEZA: Impura (escribe en pantalla el cambio de estado)
55 ;; ESTRATEGIA: Seleccion condicional mediante cond
56 ;; IMPACTO: No destructiva
57 ;; =====
58 (defun auditoria (unix-time) ;unix-time es el tiempo epoch
59   (auditoria-aux unix-time (+ unix-time 600)) ;;son 10 min en segundos
60 )
61 ;;corregi haciendo que muestre en que momento hubo una transicion
62
63 ;; =====
64 ;; FUNCION: auditoria-aux
65 ;; NATURALEZA: Impura (escribe en pantalla el cambio de estado)
66 ;; ESTRATEGIA: Recursividad simple
67 ;; IMPACTO: No destructiva
68 ;; =====
69 (defun auditoria-aux (tiempo limite) ;;hace la recursividad hasta el limite de 10 min
70   (cond
71     ((> tiempo limite) NIL)
72     (t
73      (let ((resto (mod tiempo 216)))
74        (cond
75          ((= resto 0) ;si coincide con el tiempo pasado significa que hubo un cambio de una transicion
76           (format t "Tiempo ~A: La luz a cambiado de amarillo a rojo~X" tiempo)
77           )
78          ((= resto 90)
79           (format t "Tiempo ~A: La luz a cambiado de rojo a verde~X" tiempo)
80           )
81          ((= resto 210)
82           (format t "Tiempo ~A: La luz a cambiado de verde a amarillo~X" tiempo)
83           )
84          )
85      )
86      (auditoria-aux (+ tiempo 1) limite) ;;suma de a un segundo
87    )
88  )
89 )
```

Figura 3. Código correspondiente del Requerimiento 3 en Lisp.

Fuente. Elaboración Propia.

Tal como se aprecia en la imagen de la figura 3, también se recibe como parámetro al tiempo llamándola unix-time (este mencionado es el tiempo epoch), para ir a la función “auditoria-aux” la cual va a mostrar las transiciones de los siguientes 10 min a partir de ese tiempo epoch.

y en caso de que en ese tiempo esté ocurriendo una transición de color, entonces se mostrara en terminal: Tiempo <epoch>: La luz a cambiado de <color-anterior> a <color-nuevo>, para luego pasar al siguiente segundo.

En caso de no estar ocurriendo una transición de color devuelve seguira a la proxima recursión, así continuamente hasta llegar al final. Mostrando en cual sea el caso el mensaje para los correspondientes 10 min.

## **Requerimiento 4: Análisis de Ciclos Semafóricos.**

Para la coordinación y planificar de la vía se necesita calcular cuanto dura un ciclo, siendo un ciclo rojo-verde-amarillo-rojo, para determinar la duración de un ciclo, se debe tener en cuenta la psicología del conductor, por lo cual, ciclos menores a 35 segundos y mayores a 150, se acomodan difícilmente a la mentalidad del conductor.

Por eso mismo se implementarán dos funciones, una que calcule la duración del ciclo, y otra que de una recomendación según la duración total de ese ciclo.

### **Duración del Ciclo:**

#### **Especificaciones técnicas:**

**Entrada:** Un determinado número de segundos.

**Propósito:** Determinar la duración de un ciclo completo.

```
76  ;;Requerimiento 4 a)
77  ;; -----
78  ;; Función: duracion-ciclo
79  ;; Naturaleza: Pura (sin efectos secundarios)
80  ;; Estrategia de Control: Aritmética simple (suma directa de fases)
81  ;; Impacto en Memoria: No Destructiva
82  ;; Clasificación: Función de cálculo de duración de ciclo semafórico
83  ;; -----
84
85  |
86  (defun duracion-ciclo (Seg-rojo Seg-amarillo Seg-verde)
87  |   (+ Seg-rojo Seg-amarillo Seg-verde Seg-rojo)
88  )
89
```

*Figura 4. Código Correspondiente del Requerimiento 4.a.*

*Fuente. Elaboración Propia.*

Como se muestra en la imagen, la función recibe la duración de los 3 colores, y calcula la duración del ciclo, devolviendola tal cual.

Como el requerimiento dice que la entrada es una cantidad de segundos, se da por hecho que esa entrada siempre van a ser segundos.

### **Recomendación de ciclo:**

#### **Especificaciones técnicas:**

**Entrada:** Duración calculada del ciclo.

**Salida:** Recomendación de optimización basada en estándares de la ingeniería de tráfico.



**Consideración Psicológica:** Evaluar si la duración está en el rango óptimo (35 a 150 segundos).

```
90 ;;Requerimiento 4 b)
91 ;; -----
92 ;; Función: recomendacion-ciclo
93 ;; Naturaleza: Pura (sin efectos secundarios)
94 ;; Estrategia de Control: Condicional Simple (clasificación de valores)
95 ;; Impacto en Memoria: No Destructiva (devuelve nuevo valor)
96 ;; Clasificación: Función auxiliar de recomendación sobre duración de ciclo semafórico
97 ;; -----
98
99
100
101 (defun recomendacion-ciclo (total)
102   (cond
103     ((< total 35) (list total "demasiado corto"))
104     ((> total 150) (list total "demasiado largo"))
105     (t (list total "optimo")))
106   )
107 )
108
```

Figura 4.1. Código Correspondiente al Requerimiento 4.b.

Fuente. Elaboración Propia.

Como se muestra en la figura 4.1. La función recibe la duración total del ciclo, y a partir de ello evalúa si está en un rango óptimo, si no lo estuviese, devuelve en qué situación se encuentra el ciclo actual.

Al ser el resultado de la función anterior se toma como que siempre recibe la duración total de segundos de un ciclo.

## **Requerimiento 5: Planificación Temporal.**

Se necesita calcular con objetivo de planificación y coordinación cuántos ciclos se cumplen en una determinada cantidad de minutos.

Por ello se creará una función que realice ese cálculo.

### **Especificaciones técnicas:**

**Entrada:** Determinada cantidad de segundos.

**Salida:** Número de ciclos realizados en ese tiempo.

**Aplicación:** Planificación de mantenimiento, y análisis de capacidad.

```
110 ;;Requerimiento N°5
111
112 ;; =====
113 ;; FUNCIÓN: crear-ciclos
114 ;; NATURALEZA: Pura (Retorna la cantidad de ciclos completos)
115 ;; ESTRATEGIA: Transformación aritmetica (Convierte minutos a segundos y calcula ciclos completos mediante floor)
116 ;; IMPACTO: No destructiva (No modifica variables ni estructuras)
117 ;; PROPOSITO: Determinar cuantos ciclos completos de 216 segundos pueden realizarse en un intervalo de tiempo dado.
118 ;; =====
119
120 (defun ciclos-por-tiempo (minutos)
121   (floor (/ (* minutos 60) 216))
122 )
123
```

*Figura 5. Código correspondiente a requerimiento 5.*

*Fuente. Elaboración Propia.*

Como se ve en la Figura 5, el código es sencillo, se le pasa la cantidad de minutos, los pasa a segundos, y allí hace la división correspondiente para obtener la cantidad de ciclos. Redondeando hacia abajo para que salga en número entero y diga exactamente cuántos ciclos completos.

## **Requerimiento 6: Informe de Distribución temporal.**

Por planificación logística, se necesita un informe que indique el porcentaje de cada color en una hora, bajo ciertas reglas de negocio o según las actuales.

Para ello se decidió con el equipo crear una función que permita saber el porcentaje de cada color en un intervalo de tiempo pasado en minutos.

### **Especificaciones técnicas:**

**Salida:** Porcentaje de tiempo para rojo, amarillo y verde.

**Propósito:** Optimizar el flujo vehicular y análisis de congestión.

```
138 ;;Requerimiento N°6
139
140 ;; =====
141 ;; FUNCIÓN: informe-distribucion
142 ;; NATURALEZA: Impura (Además de calcular porcentajes, muestra resultados por pantalla mediante format)
143 ;; ESTRATEGIA: Calculo directo (Utiliza funciones y operaciones aritmeticas para distribuir el tiempo entre los estados del semaforo)
144 ;; IMPACTO: No destructiva (No modifica variables externas ni estructuras de datos)
145 ;; PROPOSITO: Calcular y mostrar el porcentaje de tiempo que el semaforo permanece en rojo, verde y amarillo
146 ;; =====
```

Figura 6. Código correspondiente de Requerimiento 6.

Fuente. Elaboración Propia.

```
148 (defun informe-distribucion (minutos)
149   ;; hacemos una validacion para evitar errores de calculo
150   (if (or (not (numberp minutos)) (<= minutos 0))
151       (format t "~%La cantidad de minutos debe ser mayor a 0.-%")
152       ;; usamos let* para que se evalúe línea por línea y no todo el bloque al mismo tiempo
153       (let* (
154         ;; convertimos los minutos a segundos
155         (total-segundos (* minutos 60.0))
156
157         ;; calculamos la cantidad de ciclos completos usando la funcion del Requerimiento 5
158         (ciclos (ciclos-por-tiempo minutos))
159
160         ;; guarda los segundos que sobran despues de los ciclos completos con mod
161         (segundos-sobra (mod (* minutos 60) 216))
162
163         ;; el rojo usa 90 segundos del sobrante
164         ;; min agarra el valor mas chico, para qu si sobra menos de 90 lo agarre todo, y si sobra mas, que solo agarre 90
165         (sobra-rojo (min segundos-sobra 90))
166
167         ;; restamos el tiempo usado por el rojo para darle sobrante al verde
168         ;; max agarra el valor mas grande para asegurarnos de que no agarre un numero en negativo en caso de que ya no sobre nada
169         (sobra-verde (min (max 0 (- segundos-sobra 90)) 120))
170
171         ;; al sobrante le restamos el tiempo que usaron el rojo y el verde
172         (sobra-amarillo (max 0 (- segundos-sobra 210))))
173
174         ;; calculamos los segundos totales de cada color
175         (segundos-rojo (+ (* ciclos 90) sobra-rojo))
176         (segundos-verde (+ (* ciclos 120) sobra-verde))
177         (segundos-amarillo (+ (* ciclos 6) sobra-amarillo))
178
179         ;; calculamos los porcentajes
180         (porcentaje-rojo (* (/ segundos-rojo total-segundos) 100))
181         (porcentaje-verde (* (/ segundos-verde total-segundos) 100))
182         (porcentaje-amarillo (* (/ segundos-amarillo total-segundos) 100)))
183
184         ;; mostramos
185         (format t "~%Informe de Distribución Temporal (~A minutos):~%~%" minutos)
186         (format t "Porcentaje Rojo: ~,2F%-%" porcentaje-rojo)
187         (format t "Porcentaje Verde: ~,2F%-%" porcentaje-verde)
188         (format t "Porcentaje Amarillo: ~,2F%-%" porcentaje-amarillo))))
189
```

Figura 6.1, Código correspondiente del Requerimiento 6, segunda parte.  
Fuente. Elaboración propia.

Como se ve en la figura 6, recibe una cantidad de minutos no nula, y a partir de allí calcula el porcentaje de cada color.

## **Requerimiento 7: Aseguramiento de Calidad**

El equipo de control de calidad ha solicitado para cada uno de los requerimientos ejemplos que muestran el funcionamiento normal, ejemplos alternativos y ejemplos que generan errores.

Por ello se dejan los siguientes ejemplos para prueba del funcionamiento:

### **Requerimiento 1**

#### **Funcionamiento normal:**

(transicion 'en-rojo 'verde) ==> (EN-ROJO CAMBIAR-A-VERDE)  
(transicion 'en-verde 'amarillo) ==> (EN-VERDE  
CAMBIAR-A-AMARILLO)  
(transicion 'en-amarillo 'rojo) ==> (EN-AMARILLO  
CAMBIAR-A-ROJO)

#### **Camino alternativo (transición inválida):**

(transicion 'en-rojo 'amarillo) ==> (EN-ROJO ACCION-POR-DEFECTO)  
(transicion 'en-verde 'rojo) ==> (EN-VERDE  
ACCION-POR-DEFECTO)  
(transicion 'en-amarillo 'verde) ==> (EN-AMARILLO  
ACCION-POR-DEFECTO)

#### **Entradas no válidas:**

(transicion en-rojo 'verde) ==> variable EN-ROJO has no value  
(transicion en-verde 'amarillo) ==> variable EN-VERDE has no value

### **Requerimiento 2**

#### **Funcionamiento normal:**

(timer 50) ==> (ROJO)  
(timer 150) ==> (VERDE)  
(timer 215) ==> (AMARILLO)

**Entradas no válidas:**

(timer rojo) => variable ROJO has no value  
(timer "50") => "50" is not a real number

**Requerimiento 3****Funcionamiento normal:**

(auditoria 0) => Tiempo 0: La luz ha cambiado de amarillo a rojo  
(auditoria 90) => Tiempo 90: La luz ha cambiado de rojo a verde  
(auditoria 210) => Tiempo 210: La luz ha cambiado de verde a amarillo

**Camino alternativo (sin transición en ese instante):**

(auditoria 50) => (sin salida visible — no hay cambio)  
(auditoria 100) => (sin salida visible)

**Entradas no válidas:**

(auditoria verde) => variable VERDE has no value  
(auditoria "90") => "90" is not a real number

**Requerimiento 4****Funcionamiento normal:**

(duracion-ciclo 90 6 120) => 216  
(duracion-ciclo 60 6 120) => 186  
(duracion-ciclo 30 6 120) => 156  
(recomendacion-ciclo 90) => (90 "optimo")

**Camino alternativo:**

(recomendacion-ciclo 20) => (20 "demasiado corto")  
(recomendacion-ciclo 200) => (200 "demasiado largo")

**Entradas no válidas:**

(duración-ciclo "90" 6 120) => "90" is not a number  
(recomendación-ciclo nil) => NIL is not a real number

## Requerimiento 5

### Funcionamiento normal:

(ciclos-por-tiempo 2) => 0 (120 segundos restantes)

(ciclos-por-tiempo 5) => 1 (84 segundos restantes)

(ciclos-por-tiempo 15) => 4 (36 segundos restantes)

### Entradas no válidas:

(ciclos-por-tiempo "15") => "15" is not a number

(ciclos-por-tiempo nil) => NIL is not a number

## Requerimiento 6

### Funcionamiento normal:

(informe-distribución 1)

Informe de Distribución Temporal (1 minutos):

Porcentaje Rojo: 100.00%

Porcentaje Verde: 0.00%

Porcentaje Amarillo: 0.00%

(informe-distribución 60)

Informe de Distribución Temporal (60 minutos):

Porcentaje Rojo: 42.50%

Porcentaje Verde: 54.83%

Porcentaje Amarillo: 2.67%

### Entradas no válidas:

(informe-distribución 0) => La cantidad de minutos debe ser mayor a 0.

(informe-distribución -5) => La cantidad de minutos debe ser mayor a 0.

## Función Informe

### Funcionamiento normal:

(informe '((0 "rojo-intermitente" "rojo")

(90 "rojo" "verde-intermitente")

(93 "verde-intermitente" "verde"))))

**Salida:****Informe de Ejecución del Sistema Semafórico**

=====

Tiempo 0: La luz ha cambiado de rojo-intermitente a rojo

Tiempo 90: La luz ha cambiado de rojo a verde-intermitente

Tiempo 93: La luz ha cambiado de verde-intermitente a verde

--- Fin del Informe ---

Los datos se construyen como una lista de información, cada lista de tres elementos, el tiempo, el color anterior y el color nuevo. Esta estructura es la que se espera para escribir correctamente cada línea en el archivo.

## Bitácora de depuración:

Durante el desarrollo de los requerimientos tuvimos errores lógicos que hace falta documentar.

### Error de Transición:

Uno de los primeros errores que tuvimos fue al hacer la función “transición” que no compilaba:

```
Break 2 [5]> (transicion 'en-amarillo 'rojo)

*** - EVAL: la función T no está definida
Es posible continuar en los siguientes puntos:
USE-VALUE      :R1      Input a value to be used instead of (FDEFINITION 'T).
RETRY          :R2      Reintentar
STORE-VALUE    :R3      Input a new value for (FDEFINITION 'T).
ABORT          :R4      Abort debug loop
ABORT          :R5      Abort debug loop
ABORT          :R6      Abort main loop
```

Figura 6.2. Muestra de fallo de ejecución de función transicion

Fuente. Elaboración Propia

Esto se debió a que a la hora de escribir la función, no se cerró un paréntesis en el lugar correcto.

Lo que causaba, que el caso por defecto entre en la condición cuando resultaba ‘en-amarillo ‘rojo, haciendo que intente hacer de función a “t” siendo un carácter especial.

Esto se solucionó cerrando el paréntesis debidamente.

```
10 (defun transicion (color-actual cambiar-a)
11   (cond
12     ((and (eq color-actual 'en-rojo) (eq cambiar-a 'verde)) ;;cambie varias veces el orden del que hacia el cambio de color
13       (list color-actual 'Cambiar-a-verde)
14     )
15     ((and (eq color-actual 'en-verde) (eq cambiar-a 'amarillo))
16       (list color-actual 'Cambiar-a-amarillo)
17     )
18     ((and (eq color-actual 'en-amarillo) (eq cambiar-a 'rojo))
19       (list color-actual 'Cambiar-a-rojo)
20     )
21     (t
22       (list color-actual 'Accion-por-defecto) ;;Es en caso de que no se cumpla la validación de "cambiar-a"
23       ;;Ya que se da por hecho que el "color-actual" ingresa sin inconvenientes
24       ;;Mantiene robusto al código
25     )
26   )
27 )
```

Figura 6.3. Muestra de código corregido de Función transición

Fuente. Elaboración Propia.



## Error en auditoría:

A la hora de hacer la auditoría el primer planteamiento fue hacer la función para que reproduzca una transición de luces, lo cual en si no es muy útil.

```
Break 2 [5]> (defun auditoria (unix-time) ;;unix-time es el tiempo epoch
  (let ((resto (mod unix-time 216)))
    (cond
      ((= resto 0) ;;si coincide con el tiempo pasado significa que hubo un cambio de una transicion
        (format t "Tiempo ~A: La luz a cambiado de amarillo a rojo" unix-time)
      )
      ((= resto 90)
        (format t "Tiempo ~A: La luz a cambiado de rojo a verde" unix-time)
      )
      ((= resto 210)
        (format t "Tiempo ~A: La luz a cambiado de verde a amarillo" unix-time)
      )
    )
  )
)

AUDITORIA
Break 2 [5]> (auditoria 90)
Tiempo 90: La luz a cambiado de rojo a verde
NIL
Break 2 [5]> |
```

Figura 7. Muestra de error en función auditoría.

Fuente. Elaboración Propia.

Así que se la modifiqué para que reproduzca las transiciones a lo largo de 10 min, haciendo una suma de un segundo en una función recursiva llamada “auditoria-aux”.

```
69 > (defun auditoria-aux (tiempo limite) ;;hace la recursividad hasta el limite de 10 min
70 > (cond
71 >   (> tiempo limite) NIL)
72 > (t
73 >   (let ((resto (mod tiempo 216)))
74 >     (cond
75 >       ((= resto 0) ;;si coincide con el tiempo pasado significa que hubo un cambio de una transicion
76 >         (format t "Tiempo ~A: La luz a cambiado de amarillo a rojo~%" tiempo)
77 >       )
78 >       ((= resto 90)
79 >         (format t "Tiempo ~A: La luz a cambiado de rojo a verde~%" tiempo)
80 >       )
81 >       ((= resto 210)
82 >         (format t "Tiempo ~A: La luz a cambiado de verde a amarillo~%" tiempo)
83 >       )
84 >     )
85 >   )
86 >   (auditoria-aux (+ tiempo 1) limite) ;;suma de a un segundo
87 > )
88 > )
89 > )
```

Figura 7.1, Muestra del código de función “auditoria-aux”

Fuente. Elaboración Propia.

Como se muestra en la figura 7.1, se realiza en terminal que transición de color hubo, y luego pasa al siguiente segundo.

## Error a la hora de compilar auditoría:

En un principio al querer ejecutar el código anterior, nos devuelve error:

```
* (auditoria 1781558741)

debugger invoked on a UNBOUND-VARIABLE @1000C49D67 in thread
#<THREAD tid=9468 "main thread" RUNNING {1103FD8143}>:
  The variable TIMEPO is unbound.

Type HELP for debugger help, or (SB-EXT:EXIT) to exit from SBCL.

restarts (invokable by number or by possibly-abbreviated name):
  0: [CONTINUE ] Retry using TIMEPO.
  1: [USE-VALUE ] Use specified value.
  2: [STORE-VALUE] Set specified value and use it.
  3: [ABORT     ] Exit debugger, returning to top level.

(AUDITORIA-AUX 1781558802 1781559341)
  source: (FORMAT T "Tiempo ~A: La luz a cambiado de rojo a verde~%" TIMEPO)
0] |
```

Figura 7.1.1 Muestra de error de ejecución de función auditoria

Fuente. Elaboración Propia

Esto se debía a que en la función el parámetro “tiempo” estaba mal escrito y decía “tiempo”, lo que llevaba a ese error a la hora de querer ejecutar el código. Esto se soluciona simplemente corrigiendo el nombre del parámetro, por cómo debería ser.

```
69 (defun auditoria-aux (tiempo limite) ;;hace la recursividad hasta el limite de 10 min
70   (cond
71     ((> tiempo limite) NIL)
72     (t
73      (let ((resto (mod tiempo 216)))
74        (cond
75          ((= resto 0) ;;si coincide con el tiempo pasado significa que hubo un cambio de una transicion
76           (format t "Tiempo ~A: La luz a cambiado de amarillo a rojo~%" tiempo)
77           )
78          ((= resto 90)
79           (format t "Tiempo ~A: La luz a cambiado de rojo a verde~%" tiempo)
80           )
81          ((= resto 210)
82           (format t "Tiempo ~A: La luz a cambiado de verde a amarillo~%" tiempo)
83           )
84        )
85      )
86      (auditoria-aux (+ tiempo 1) limite) ;;suma de a un segundo ;;dando lugar a la recursividad de cola
87    )
88  )
89 )
```

Figura 7.1.2 Muestra del código corregido del req 3.

Fuente. Elaboración Propia

## Error de Ciclos de semáforo:

A la hora de hacer el req 1, 2 y 3, el enunciado que se nos daba, era respecto a el ciclo “rojo-amarillo-verde”.

```
Break 2 [5]> (defun transicion (color-actual cambiar-a)
  (cond
    ((and (eq color-actual 'en-rojo) (eq cambiar-a 'amarillo)) ;;cambie varias veces el orden del que hacia el cambio de color
      (list color-actual 'Cambiar-a-amarillo)
    )
    ((and (eq color-actual 'en-amarillo) (eq cambiar-a 'verde))
      (list color-actual 'Cambiar-a-verde)
    )
    ((and (eq color-actual 'en-verde) (eq cambiar-a 'rojo))
      (list color-actual 'Cambiar-a-rojo)
    )
    (t
      (list color-actual 'Accion-por-defecto) ;;Es en caso de que no se cumpla la validación de "cambiar-a"
      ;;Ya que se da por hecho que el "color-actual ingresa sin inconvenientes"
      ;;Mantiene robuzto al codigo
    )
  )
)
```

Figura 7.2.1 Muestra del código de Requerimiento 1 de manera errónea.

Fuente. Elaboración Propia

```
(defun timer (unix-time) ;;unix-time es el tiempo epoch
  (let ((resto (mod unix-time 216))) ;;coloque un let para evitar la repeticion
    (cond
      ((< resto 90) 'rojo)
      ((< resto 96) 'amarillo)
      (t
        'verde
      )
    )
  )
)
```

Figura 7.2.2 Muestra del Código del Requerimiento 2 de manera errónea

Fuente. Elaboración Propia.

```
(defun auditoria-aux (tiempo limite) ;;hace la recursividad hasta el limite de 10 min
  (cond
    ((> tiempo limite) NIL)
    (t
      (let ((resto (mod tiempo 216)))
        (cond
          ((= resto 0) ;;si coincide con el tiempo pasado significa que hubo un cambio de una transicion
            (format t "Tiempo ~A: La luz a cambiado de amarillo a rojo~%" tiempo)
          )
          ((= resto 90)
            (format t "Tiempo ~A: La luz a cambiado de rojo a amarillo~%" tiempo)
          )
          ((= resto 96)
            (format t "Tiempo ~A: La luz a cambiado de amarillo a verde~%" tiempo)
          )
        )
      )
      (auditoria-aux (+ tiempo 1) limite) ;;suma de a un segundo
    )
  )
)
```

Figura 7.2.3 Muestra de Código de función aux del Requerimiento 3 de manera errónea.

Fuente. Elaboración Propia.

Conceptualmente lo íbamos a cumplir, pero de por si no nos gustaba por la logica que seguía, entonces interpretando la ley 24.449 de seguridad vial en argentina, el ciclo correcto es “rojo-verde-amarillo”, que seguidamente, el

profesor corrigio que era un error de tipeo.  
Por ello debimos corregir las funciones que ya habíamos planteado.

### Error de duración de ciclo:

Al crear la función duración-ciclo, originalmente estaba sumando de más una duración, ya que habíamos seguido el tipeo del archivo del trabajo práctico:

```
(defun duracion-ciclo (Seg-rojo Seg-amarillo Seg-verde)
  (+ Seg-rojo Seg-amarillo Seg-verde Seg-rojo)
)
```

Figura 7.3 Muestra de código erróneo de Requerimiento 4.a  
Fuente. Elaboración Propia.

Esto fue una simple corrección de borrar la duración sobrante del color rojo.

### Error de interpretación de Requerimiento 4:

Del requerimiento ya mencionado, antes de este error al escribirlo tuvimos el problema de interpretar la función de recomendación y suma, juntas.

```
(defun duracion-ciclo (Seg-rojo Seg-amarillo Seg-verde)
  (recomendacion-ciclo (+ Seg-rojo Seg-amarillo Seg-verde Seg-rojo))
)
```

Figura 7.4. Muestra del Código duracion-ciclo del requerimiento 4 erróneo.  
Fuente. Elaboración Propia.

Como se ve en la figura 7.4 el error era poner la recomendación dentro de la función que calcula la duración del ciclo, además del error anteriormente mencionado.

Quedando solucionado al borrar la duración de más y dejando que la función haga solo el cálculo.

Como se muestra en la figura 4.

### Error de overflow:

Al principio la función auditoria-aux estaba pensada para una hora, es decir, 3600 segundos. Pero eso provocaba un overflow:

```
[2]> (defun auditoria (unix-time) ;unix-time es el tiempo epoch
      (auditoria-aux unix-time (+ unix-time 3600)) ;;son 10 min en segundos
    )
```

Figura 7.5. Muestra del Código de auditoria del requerimiento 3 erróneo.  
Fuente. Elaboración Propia.

```
[4]> (auditoria 1781573567)
Tiempo 1781573613: La luz a cambiado de verde a amarillo-intermitente
Tiempo 1781573616: La luz a cambiado de amarillo-intermitente a amarillo
Tiempo 1781573622: La luz a cambiado de amarillo a rojo-intermitente
Tiempo 1781573625: La luz a cambiado de rojo-intermitente a rojo
Tiempo 1781573715: La luz a cambiado de rojo a verde-intermitente
Tiempo 1781573718: La luz a cambiado de verde-intermitente a verde
Tiempo 1781573838: La luz a cambiado de verde a amarillo-intermitente
Tiempo 1781573841: La luz a cambiado de amarillo-intermitente a amarillo
Tiempo 1781573847: La luz a cambiado de amarillo a rojo-intermitente
Tiempo 1781573850: La luz a cambiado de rojo-intermitente a rojo
Tiempo 1781573940: La luz a cambiado de rojo a verde-intermitente
Tiempo 1781573943: La luz a cambiado de verde-intermitente a verde
Tiempo 1781574063: La luz a cambiado de verde a amarillo-intermitente
Tiempo 1781574066: La luz a cambiado de amarillo-intermitente a amarillo
Tiempo 1781574072: La luz a cambiado de amarillo a rojo-intermitente
Tiempo 1781574075: La luz a cambiado de rojo-intermitente a rojo
Tiempo 1781574165: La luz a cambiado de rojo a verde-intermitente
Tiempo 1781574168: La luz a cambiado de verde-intermitente a verde
Tiempo 1781574288: La luz a cambiado de verde a amarillo-intermitente
Tiempo 1781574291: La luz a cambiado de amarillo-intermitente a amarillo
Tiempo 1781574297: La luz a cambiado de amarillo a rojo-intermitente
Tiempo 1781574300: La luz a cambiado de rojo-intermitente a rojo
Tiempo 1781574390: La luz a cambiado de rojo a verde-intermitente
```

*Figura 7.6. Muestra de principio de ejecución de auditoria.*

*Fuente. Elaboración Propia.*

```

Tiempo 1781575088: La luz a cambiado de verde-intermitente a verde
Tiempo 1781575188: La luz a cambiado de verde a amarillo-intermitente
Tiempo 1781575191: La luz a cambiado de amarillo-intermitente a amarillo
Tiempo 1781575197: La luz a cambiado de amarillo a rojo-intermitente
Tiempo 1781575200: La luz a cambiado de rojo-intermitente a rojo
Tiempo 1781575290: La luz a cambiado de rojo a verde-intermitente
Tiempo 1781575293: La luz a cambiado de verde-intermitente a verde
Tiempo 1781575413: La luz a cambiado de verde a amarillo-intermitente
Tiempo 1781575416: La luz a cambiado de amarillo-intermitente a amarillo
Tiempo 1781575422: La luz a cambiado de amarillo a rojo-intermitente
Tiempo 1781575425: La luz a cambiado de rojo-intermitente a rojo
Tiempo 1781575515: La luz a cambiado de rojo a verde-intermitente
Tiempo 1781575518: La luz a cambiado de verde-intermitente a verde
Tiempo 1781575638: La luz a cambiado de verde a amarillo-intermitente
Tiempo 1781575641: La luz a cambiado de amarillo-intermitente a amarillo
Tiempo 1781575647: La luz a cambiado de amarillo a rojo-intermitente
Tiempo 1781575650: La luz a cambiado de rojo-intermitente a rojo
Tiempo 1781575740: La luz a cambiado de rojo a verde-intermitente
Tiempo 1781575743: La luz a cambiado de verde-intermitente a verde
Tiempo 1781575863: La luz a cambiado de verde a amarillo-intermitente
Tiempo 1781575866: La luz a cambiado de amarillo-intermitente a amarillo
Tiempo 1781575872: La luz a cambiado de amarillo a rojo-intermitente
Tiempo 1781575875: La luz a cambiado de rojo-intermitente a rojo
Tiempo 1781575965: La luz a cambiado de rojo a verde-intermitente
Tiempo 1781575968: La luz a cambiado de verde-intermitente a verde
Tiempo 1781576088: La luz a cambiado de verde a amarillo-intermitente
Tiempo 1781576091: La luz a cambiado de amarillo-intermitente a amarillo
Tiempo 1781576097: La luz a cambiado de amarillo a rojo-intermitente
Tiempo 1781576100: La luz a cambiado de rojo-intermitente a rojo
Tiempo 1781576190: La luz a cambiado de rojo a verde-intermitente
Tiempo 1781576193: La luz a cambiado de verde-intermitente a verde
Tiempo 1781576313: La luz a cambiado de verde a amarillo-intermitente
Tiempo 1781576316: La luz a cambiado de amarillo-intermitente a amarillo
Tiempo 1781576322: La luz a cambiado de amarillo a rojo-intermitente
Tiempo 1781576325: La luz a cambiado de rojo-intermitente a rojo
Tiempo 1781576415: La luz a cambiado de rojo a verde-intermitente
Tiempo 1781576418: La luz a cambiado de verde-intermitente a verde
Tiempo 1781576538: La luz a cambiado de verde a amarillo-intermitente
Tiempo 1781576541: La luz a cambiado de amarillo-intermitente a amarillo
Tiempo 1781576547: La luz a cambiado de amarillo a rojo-intermitente
Tiempo 1781576550: La luz a cambiado de rojo-intermitente a rojo
Tiempo 1781576640: La luz a cambiado de rojo a verde-intermitente
Tiempo 1781576643: La luz a cambiado de verde-intermitente a verde
Tiempo 1781576763: La luz a cambiado de verde a amarillo-intermitente
Tiempo 1781576766: La luz a cambiado de amarillo-intermitente a amarillo
Tiempo 1781576772: La luz a cambiado de amarillo a rojo-intermitente
Tiempo 1781576775: La luz a cambiado de rojo-intermitente a rojo
Tiempo 1781576865: La luz a cambiado de rojo a verde-intermitente
Tiempo 1781576868: La luz a cambiado de verde-intermitente a verde

*** _ Program stack overflow. RESET

```

Figura 7.6. Muestra de final de ejecución de auditoria.

Fuente. Elaboración Propia.

Como se ve en la imagen, al intentar hacer con tantos llega a un overflow, lo que se decidió hacer entonces es que sea en un intervalo más reducido, es decir 10 minutos.

Ya que sirve para una reconstrucción histórica,

## Iteración 2:

Con el fin de tener una intermitencia de seguridad y una persistencia de datos de se hace una extensión por medio comentado del sistema ya hecho.

En el cual se modifican las funciones para tener un tiempo de intermitencia entre cada luz, con el ejemplo de entre rojo y verde, hay una intermitencia del color rojo, agregando 3 segundos más, y así con todos.

Y con el objetivo de tener una persistencia de datos, se guardan en texto plano los cambios de estados del semáforo.

## Actualización de requerimientos:

### Requerimiento 1:

```
1 ;;Requerimiento 1 con Iteracion 2 Extencion 1
2
3 ;; -----
4 ;; FUNCION: transicion
5 ;; NATURALEZA: Pura (Devuelve la lista con la transicion de colores realiza)
6 ;; ESTRATEGIA: Selecccion condicional mediante cond
7 ;; IMPACTO: No destructiva
8 ;; -----
9
10 (defun transicion (color-actual cambiar-a)
11   (cond
12     ((and (eq color-actual 'en-rojo) (eq cambiar-a 'verde-intermitente))
13      (list color-actual 'Cambiar-a-verde-intermitente))
14     )
15     ((and (eq color-actual 'en-verde-intermitente) (eq cambiar-a 'verde))
16      (list color-actual 'Cambiar-a-verde))
17     )
18     ((and (eq color-actual 'en-verde) (eq cambiar-a 'amarillo-intermitente))
19      (list color-actual 'Cambiar-a-amarillo-intermitente))
20     )
21     ((and (eq color-actual 'en-amarillo-intermitente) (eq cambiar-a 'amarillo))
22      (list color-actual 'Cambiar-a-amarillo))
23     )
24     ((and (eq color-actual 'en-amarillo) (eq cambiar-a 'rojo-intermitente))
25      (list color-actual 'Cambiar-a-rojo-intermitente))
26     )
27     ((and (eq color-actual 'en-rojo-intermitente) (eq cambiar-a 'rojo))
28      (list color-actual 'Cambiar-a-rojo))
29     )
30     (t
31      (list color-actual 'Accion-por-defecto))
32     )
33   )
34 )
35 )
```

Figura 8.1. Código de función transicion pasado a la iteración 2.

Fuente. Elaboración Propia.

Como se ve en la figura 8.1, se agrego todas las intermitencias a las transiciones, es decir, se agregaron las transiciones de: verde-intermitente, amarillo-intermitente y rojo-intermitente. Cada una de ellas antes de cambiar a su respectivo color

## Requerimiento 2:

```
39 ;;Requerimiento 2 con Iteracion 2 Extencion 1
40
41 ;; =====
42 ;; FUNCION: timer
43 ;; NATURALEZA: Pura (no devuelve nada en pantalla)
44 ;; ESTRATEGIA: Selecccion condicional
45 ;; IMPACTO: No destructiva
46 ;; =====
47 (defun timer (unix-time)
48   (let ((resto (mod unix-time 225))) ;;coloque un let para evitar la repeticion
49     (cond
50       ((< resto 90) 'rojo)
51       ((< resto 93) 'verde-intermitente)
52       ((< resto 213) 'verde)
53       ((< resto 216) 'amarillo-intermitente)
54       ((< resto 222) 'amarillo)
55       (t 'rojo-intermitente)
56     )
57   )
58 )
```

Figura 8.2. Código de función timer en iteración 2.

Fuente. Elaboración Propia.

Como se muestra en la imagen, se agregó en la función cuando está en los tiempos de verde-intermitente, amarillo-intermitente y rojo-intermitente para el nuevo ciclo.

## Requerimiento 3:

```
99 ;; =====
100 ;; FUNCION: auditoria-aux
101 ;; NATURALEZA: Impura (escribe en pantalla el cambio de estado)
102 ;; ESTRATEGIA: Recursividad de cola
103 ;; IMPACTO: No destructiva
104 ;; =====
105 (defun auditoria-aux (tiempo limite) ;;hace la recursividad hasta el limite de 10 min
106   (cond
107     ((> tiempo limite) NIL)
108     (t
109      (let ((resto (mod tiempo 225)))
110        (cond
111          ((= resto 0) ;;si coincide con el tiempo pasado significa que hubo un cambio de una transicion
112           (format t "Tiempo ~A: La luz a cambiado de rojo-intermitente a rojo-~%" tiempo)
113         )
114          ((= resto 90)
115           (format t "Tiempo ~A: La luz a cambiado de rojo a verde-intermitente-~%" tiempo)
116         )
117          ((= resto 93)
118           (format t "Tiempo ~A: La luz a cambiado de verde-intermitente a verde-~%" tiempo)
119         )
120          ((= resto 213)
121           (format t "Tiempo ~A: La luz a cambiado de verde a amarillo-intermitente-~%" tiempo)
122         )
123          ((= resto 216)
124           (format t "Tiempo ~A: La luz a cambiado de amarillo-intermitente a amarillo-~%" tiempo)
125         )
126          ((= resto 222)
127           (format t "Tiempo ~A: La luz a cambiado de amarillo a rojo-intermitente ~%" tiempo)
128         )
129        )
130      )
131      (auditoria-aux (+ tiempo 1) limite) ;;va sumando de a 1 segundo
132    )
133  )
134 )
```

Figura 8.3. Código de función auditoria-aux en iteración 2.

Fuente. Elaboración Propia.

Como se muestra en la imagen el cambio que se realizó en la auditoria, fue únicamente en la auditoria-aux, donde se agregaron los mensajes para los cambios a los intermitentes en el tiempo del nuevo ciclo.



## Requerimiento 4:

```
136 ;;Requerimiento 4 a) con Iteracion 2 Extencion 1
137 ;; -----
138 ;; Función: duracion-ciclo
139 ;; Naturaleza: Pura (sin efectos secundarios)
140 ;; Estrategia de Control: Aritmética simple (suma directa de fases)
141 ;; Impacto en Memoria: No Destructiva
142 ;; Clasificación: Función de cálculo de duración de ciclo semafórico
143 ;; -----
144
145 ;;la logica aqui consta en ingresar los segundos de cada color del semaforo rojo, amarillo y verde, e
146 (defun duracion-ciclo (Seg-rojo Seg-amarillo Seg-verde Seg-intermitencia)
147   (+ Seg-rojo Seg-amarillo Seg-verde (* 3 Seg-intermitencia))
148 )
149
```

Figura 8.4. Código de función *duracion-ciclo* en iteración 2.

Fuente. Elaboración Propia.

Para el requerimiento 4, la modificación fue para la función “*duracion-ciclo*”, añadiendo los 3 tiempos de los intermitentes.

El otro ítem del requerimiento 4, no sufrió cambios al ser una recomendación.

## Requerimiento 5:

```
170 ;;Requerimiento N°5 con Iteracion 2 Extencion 1
171
172 ;; =====
173 ;; FUNCIÓN: ciclos-por-tiempo
174 ;; NATURALEZA: Pura (Retorna la cantidad de ciclos completos)
175 ;; ESTRATEGIA: Transformacion aritmetica (Convierte minutos a segundos y calcula ciclos completos mediante floor)
176 ;; IMPACTO: No destructiva (No modifica variables ni estructuras)
177 ;; PROPOSITO: Determinar cuantos ciclos completos de 225 segundos pueden realizarse en un intervalo de tiempo dado.
178 ;; =====
179
180 (defun ciclos-por-tiempo (minutos)
181   (floor (* minutos 60) 225))
182
```

Figura 8.5. Código de función *ciclos-por-tiempo* en iteración 2.

Fuente. Elaboración Propia.

En el requerimiento 5, la única modificación fue cambiar la duración del ciclo.

## Requerimiento 6:

```
193 (defun informe-distribucion (minutos)
194   ;; hacemos una validacion para evitar errores de calculo
195   (if (or (not (numberp minutos)) (<= minutos 0))
196       (format t "~%La cantidad de minutos debe ser mayor a 0.~%" )
197       ;; usamos let* para que se evalúe linea por linea y no todo el bloque al mismo tiempo
198       (let* (
199         ;; convertimos los minutos a segundos
200         (total-segundos (* minutos 60.0))
201
202         ;; calculamos la cantidad de ciclos completos usando la funcion del Requerimiento 5
203         (ciclos (ciclos-por-tiempo minutos))
204
205         ;; guarda los segundos que sobran despues de los ciclos completos con mod
206         (segundos-sobra (mod (* minutos 60) 225))
207
208         ;; el rojo usa 90 segundos del sobrante
209         ;; min agarra el valor mas chico, para qu si sobra menos de 90 lo agarre todo, y si sobra mas, que solo agarre 90
210         (sobra-rojo (min segundos-sobra 90))
211
212         ;; restamos el tiempo usado por el anterior estado para darle sobrante al verde intermitente
213         (sobra-verde-inter (min (- segundos-sobra sobra-rojo) 3))
214
215         ;; restamos el tiempo usado por los anteriores estados para darle sobrante al verde
216         (sobra-verde (min (- segundos-sobra sobra-rojo sobra-verde-inter) 120))
217
218         ;; restamos el tiempo usado por los anteriores estados para darle sobrante al amarillo intermitente
219         (sobra-amarillo-inter (min (- segundos-sobra sobra-rojo sobra-verde-inter sobra-verde) 3))
220
221         ;; restamos el tiempo usado por los anteriores estados para darle sobrante al amarillo
222         (sobra-amarillo (min (- segundos-sobra sobra-rojo sobra-verde-inter sobra-verde sobra-amarillo-inter) 6))
223
224         ;; restamos el tiempo usado por los anteriores estados para darle sobrante al rojo intermitente
225         (sobra-rojo-inter (min (- segundos-sobra sobra-rojo sobra-verde-inter sobra-verde sobra-amarillo-inter sobra-amarillo) 3))
226
227         ;; calculamos los segundos totales de cada estado
228         (segundos-rojo (+ (* ciclos 90) sobra-rojo))
229         (segundos-verde (+ (* ciclos 120) sobra-verde))
230         (segundos-amarillo (+ (* ciclos 6) sobra-amarillo))
231         (segundos-verde-inter (+ (* ciclos 3) sobra-verde-inter))
232         (segundos-amarillo-inter (+ (* ciclos 3) sobra-amarillo-inter))
233         (segundos-rojo-inter (+ (* ciclos 3) sobra-rojo-inter))
234
235         ;; calculamos los porcentajes
236         (porcentaje-rojo (* (/ segundos-rojo total-segundos) 100))
237         (porcentaje-verde (* (/ segundos-verde total-segundos) 100))
238         (porcentaje-amarillo (* (/ segundos-amarillo total-segundos) 100))
239         (porcentaje-verde-inter (* (/ segundos-verde-inter total-segundos) 100))
240         (porcentaje-amarillo-inter (* (/ segundos-amarillo-inter total-segundos) 100))
241         (porcentaje-rojo-inter (* (/ segundos-rojo-inter total-segundos) 100))
242       )
243
244       ;; mostramos los porcentajes
245       (format t "~%Informe de Distribución Temporal (~A minutos):~%~%" minutos)
246       (format t "Porcentaje Rojo: ~,2F%~%" porcentaje-rojo)
247       (format t "Porcentaje Verde Intermitente: ~,2F%~%" porcentaje-verde-inter)
248       (format t "Porcentaje Verde: ~,2F%~%" porcentaje-verde)
249       (format t "Porcentaje Amarillo Intermitente: ~,2F%~%" porcentaje-amarillo-inter)
250       (format t "Porcentaje Amarillo: ~,2F%~%" porcentaje-amarillo)
251       (format t "Porcentaje Rojo Intermitente: ~,2F%~%" porcentaje-rojo-inter))))
```

Figura 8.6. Código de función informe-distribucional en iteración 2.

Fuente. Elaboración Propia.

El cambio del Requerimiento 6 en la iteración dos, es la contabilización de los intermitentes además de los colores.

Y finalmente mostrar los porcentajes totales al final, tanto de los colores como de cada intermitente.

## Función Informe:

```
269 > (defun informe (datos)           ;;La informacion se proporciona
270 >   (with-open-file
271 >     (stream
272 >       "informe-ejecucion-semaforo.txt"
273 >       :direction :output)           ;;Se creara en la carpeta desde donde esta ejecutandose el lisp
274 >
275 >     (format stream
276 >       "Informe de Ejecucion del Sistema Semaforico~%")
277 >     (format stream "=====")
278 >     (mapcar (lambda (informacion)
279 >               (format stream "Tiempo ~A: La luz ha cambiado de ~A a ~A~%"
280 >                             (first informacion)
281 >                             (second informacion)
282 >                             (third informacion)))
283 >             datos)
284 >     (format stream "~%--- Fin del Informe ---")
285 >   )
286 > )
```

Figura 8.7. Código de función informe en iteración 2.

Fuente. Elaboración Propia.

La siguiente función fue creada con la intención de recibir las salidas de la función auditoría para cargarlas en un archivo.txt.

Así guardando los cambios de color a partir de un tiempo determinado en un texto plano.

## Fase 2: Autonomía y Ecosistema

### Quicklisp

Quicklisp es el gestor de paquetes más utilizado en Common Lisp. Su función principal es permitir instalar, descargar y cargar librerías externas de manera automática.

Suele ser usado en implementaciones o lenguajes modernos como SBCL, CLISP o Clozure CL, funciona en varios Sistemas Operativos de dispositivos digitales tales como **Linux, macOS y Windows**.

### Algunas Desventajas de su uso:

Quicklisp prioriza que las librerías compilen juntas, pero no garantiza o da fiabilidad la compatibilidad funcional entre todas.

No todas las librerías funcionan correctamente por defecto o al usarse aquí, por lo que las mismas pueden requerir configuraciones adicionales.

Quicklisp está pensado más para desarrollo y experimentación que para despliegues críticos (Es decir su uso no está inclinado tanto hacia la producción).

No destaca en tener una seguridad afianzada, aunque el instalador inicial tiene firma PGP, las descargas posteriores de librerías se hacen por **HTTP sin verificación criptográfica**. Este contexto hace propenso a ataques de tipo *man-in-the-middle* (MITM), donde un atacante en la red puede inyectar código malicioso en los paquetes

### Ventajas principales de su uso

- **Simplicidad:** Comandos cortos y uniformes.
- **Portabilidad:** Funciona en todas las implementaciones populares.
- **Actualizaciones mensuales:** Garantiza que las librerías se compilen juntas.
- **Integración con ASDF:** Se conecta con el sistema estándar de construcción de proyectos en Lisp.

### Librería Seleccionada: local-time

Para este proyecto decidimos integrar la librería “local-time”. Siendo esta una librería especializada en fechas, horarios y conversión de tiempo, nos pareció útil para la función de auditoría para mostrar en vez de el tiempo en formato epoch, mostrarlo como se nos es más común a nosotros. A la hora de empezar a usar el instalador de paquetes, nos dimos con el inconveniente de que quicklisp, está optimizado para SBCL (Steel Bank Common Lisp). Que es la implementación más moderna y usada del lenguaje Common Lisp, si bien nosotros usamos CLISP, para utilización de la librería local time, nos vimos respaldados por SBCL.

Entonces primeramente para cargar la librería ya instalada y el quicklisp activo, en un entorno de SBCL, como puede ser la terminal se realiza el comando:

*(ql:quickload "local-time")*

```
* (ql:quickload "local-time")
To load "local-time":
  Load 1 ASDF system:
    local-time
; Loading "local-time"

("local-time")
*
```

Figura 9. Ejecución de comando de quicklisp

Fuente. Elaboración propia.

Este comando, descarga la librería si no estuviera instalada y la deja disponible para el uso del sistema. Seguidamente, para la modificación de la función auditoría crearemos una función para poder ver el tiempo epoch de una mejor manera:

```
418 ;;Requerimiento 3
419 ;; =====
420 ;; FUNCION: unix-a-normal
421 ;; NATURALEZA: Pura
422 ;; ESTRATEGIA:
423 ;; IMPACTO: No destructiva
424 ;; =====
425 (defun unix-a-normal (unix-time)
426   (local-time:format-timestring nil ;;convierte en texto legible
427     (local-time:universal-to-timestamp (+ unix-time 2208988800)) ;;Transforma en formato interno de lisp, y lo hace timestamp de local-time
428     :format '(:day "/" :month "/" :year " " :hour ":" :min ":" :sec))
429 )
430
```

Figura 9.1. Función unix-a-normal

Fuente. Elaboración propia.

Aquí se muestra la función unix-a-normal, que lo que hace es, con

*local-time:format-timestring nil*

Convierte el timestamp en un tiempo legible, un timestamp es un número que representa un punto en el tiempo. Osea decir como se va a ver la fecha al final. además del NIL que es para que indica que no se guarda ni como variable, ni como archivo.

*(local-time:universal-to-timestamp (+ unix-time 2208988800))*

Esta línea lo que realiza es convertir ese número en un timestamp, que es un número que entiende local-time, y al tiempo que unix, se le suma ese número ya que es epoch usado por Lisp, y este es desde el primero de enero de 1900, mientras que el epoch tradicional es de el primero de enero de 1970, y la diferencia entre esos años en segundos, es exactamente ese número.

```
:format '(:day "/" :month "/" :year " " :hour ":" :min ":" :sec))
```

finalmente define el formato de salida, siendo day: día, month:mes, year: año, hour: hora, min: minuto, sec: segundos.

Luego de eso, se crea la función auditoria como ya se la menciono anteriormente con la diferencia de que en, la función a la que pasa el mando “auditoria-aux” se le modifica lo siguiente:

```
104 (defun auditoria-aux (tiempo limite) ;;hace la recursividad hasta el limite de 10 min
105 (cond
106   (> tiempo limite) NIL
107   (t
108     (let ((resto (mod tiempo 225)))
109       (cond
110         ((= resto 0) ;;si coincide con el tiempo pasado significa que hubo un cambio de una transicion
111          (format t "Tiempo ~A: La luz a cambiado de amarillo-intermitente a rojo-%" (unix-a-normal tiempo))
112          )
113         ((= resto 90)
114          (format t "Tiempo ~A: La luz a cambiado de rojo a verde-intermitente-%" (unix-a-normal tiempo))
115          )
116         ((= resto 93)
117          (format t "Tiempo ~A: La luz a cambiado de verde-intermitente a verde-%" (unix-a-normal tiempo))
118          )
119         ((= resto 213)
120          (format t "Tiempo ~A: La luz a cambiado de verde a amarillo-intermitente-%" (unix-a-normal tiempo))
121          )
122         ((= resto 216)
123          (format t "Tiempo ~A: La luz a cambiado de amarillo-intermitente a amarillo-%" (unix-a-normal tiempo))
124          )
125         ((= resto 222)
126          (format t "Tiempo ~A: La luz a cambiado de amarillo a rojo-intermitente-%" (unix-a-normal tiempo))
127          )
128         )
129       )
130     (auditoria-aux (+ tiempo 1) limite) ;;va sumando de a 1 segundo
131   )
132 )
133 )
```

Figura 9.2. Nueva función auditoria para iteración 2

Fuente. Elaboración Propia.

De esta manera, en vez de mostrar el tiempo en formato epoch, llama a la función unix-a-normal.

Y permitiendo así, al pasarle un tiempo epoch, devolver el tiempo en un formato legible:

```
* (auditoria 1781553600)
Tiempo 15/6/2026 20:0:0: La luz a cambiado de amarillo-intermitente a rojo
Tiempo 15/6/2026 20:1:30: La luz a cambiado de rojo a verde-intermitente
Tiempo 15/6/2026 20:1:33: La luz a cambiado de verde-intermitente a verde
Tiempo 15/6/2026 20:3:33: La luz a cambiado de verde a amarillo-intermitente
Tiempo 15/6/2026 20:3:36: La luz a cambiado de amarillo-intermitente a amarillo
Tiempo 15/6/2026 20:3:42: La luz a cambiado de amarillo a rojo-intermitente
Tiempo 15/6/2026 20:3:45: La luz a cambiado de amarillo-intermitente a rojo
Tiempo 15/6/2026 20:5:15: La luz a cambiado de rojo a verde-intermitente
Tiempo 15/6/2026 20:5:18: La luz a cambiado de verde-intermitente a verde
Tiempo 15/6/2026 20:7:18: La luz a cambiado de verde a amarillo-intermitente
Tiempo 15/6/2026 20:7:21: La luz a cambiado de amarillo-intermitente a amarillo
Tiempo 15/6/2026 20:7:27: La luz a cambiado de amarillo a rojo-intermitente
Tiempo 15/6/2026 20:7:30: La luz a cambiado de amarillo-intermitente a rojo
Tiempo 15/6/2026 20:9:0: La luz a cambiado de rojo a verde-intermitente
Tiempo 15/6/2026 20:9:3: La luz a cambiado de verde-intermitente a verde
NIL
* |
```

*Figura9.3. Muestra de ejecución de del Requerimiento 3, en la fase 2*  
*Fuente. Elaboración Propia*

Como se muestra, paso del tiempo epoch, al formato que especificamos anteriormente, sienta día/mes/año hora:minuto:segundo.

### **Fase 3: Estudio Comparativo**

#### **Lenguaje CLOJURE:**

Clojure es un lenguaje de programación de propósito general perteneciente a la familia Lisp, creado por Rich Hickey. Fue diseñado con un fuerte enfoque en la programación funcional y en la reducción de la complejidad asociada al desarrollo de aplicaciones concurrentes.

Uno de sus principales objetivos es permitir la construcción de programas más simples, seguros y fáciles de mantener mediante el uso de datos inmutables y funciones puras. A diferencia de otros lenguajes que se centran en la modificación constante del estado de los objetos, Clojure promueve la transformación de datos a través de funciones, favoreciendo un estilo de programación más predecible.

Entre sus características más importantes se encuentran la inmutabilidad por defecto, el soporte para funciones de orden superior, la evaluación interactiva



mediante REPL (Read-Eval-Print Loop) y la posibilidad de trabajar con secuencias perezosas, que permiten procesar datos únicamente cuando son necesarios.

Además, Clojure se ejecuta sobre la Máquina Virtual de Java, lo que facilita su integración con bibliotecas y aplicaciones desarrolladas en Java. Esta característica le permite aprovechar un ecosistema amplio y maduro sin perder las ventajas de la programación funcional.

Gracias a estas cualidades, Clojure es utilizado en áreas como sistemas financieros, procesamiento de datos, aplicaciones web y sistemas distribuidos, especialmente en proyectos donde la concurrencia y la confiabilidad son aspectos fundamentales. Diversas empresas reconocidas han utilizado Clojure en sus sistemas y servicios. Entre ellas se encuentran Nubank, uno de los bancos digitales más grandes de América Latina, Walmart, para el desarrollo de algunas soluciones internas, Cisco, en proyectos relacionados con infraestructura tecnológica y Datomic, empresa creadora de una base de datos diseñada por el mismo autor de Clojure.

## **RESPUESTA A PREGUNTAS TEÓRICAS:**

### **1. ¿Cuál es la ventaja técnica y de rendimiento de un Keyword en Clojure para mapear estados?**

Tanto los símbolos cotizados ('rojo) como los keywords (:rojo) pueden utilizarse para representar estados, Clojure aprovecha el uso de keywords porque ofrece ventajas de rendimiento y facilidad en la práctica.

Los keywords son valores inmutables que están internados, es decir, Clojure mantiene una instancia única de cada keyword en memoria. Esto facilita que cuando se comparan dos keywords iguales, la verificación puede realizarse mediante identidad de referencia, una operación rápida. En cambio, los símbolos no poseen este mismo comportamiento y su comparación resulta ineficiente.

Además, los keywords tienen características muy útiles: pueden utilizarse directamente como claves de mapas y también como funciones de acceso. Por ejemplo: (def estado {:color :rojo :duracion 90})

```
(:color estado)  
;; => :rojo
```

Esto permite representar los estados del semáforo claro y eficiente:

```
:rojo  
:amarillo  
:verde
```

y almacenarlos fácilmente dentro de estructuras de datos:

```
{:estado :rojo  
 :tiempo-restante 30}
```

Por estas razones, los keywords son la mejor opción para mapear estados, ya que consumen menos recursos, facilitan las comparaciones y simplifican el acceso a los datos.

## **2. Clojure prohíbe la mutabilidad por defecto usando estructuras de datos persistentes. ¿Cómo maneja Clojure el paso del tiempo en el simulador semafórico sin modificar variables?**

Clojure adopta un enfoque funcional basado en la inmutabilidad, por lo que una vez creado un valor no puede modificarse. En lugar de cambiar variables continuamente, cada operación genera un nuevo valor a partir del anterior. Para representar el paso del tiempo en un simulador semafórico no es necesario incrementar los contadores ni alterar variables globales. La función recibe el instante de tiempo actual como parámetro y calcula cuál debe ser el estado del semáforo en ese momento.

Por ejemplo:

```
(timer 0 90 6 120)  
;; => :rojo
```

```
(timer 100 90 6 120)
;; => :verde
```

```
(timer 200 90 6 120)
=> :rojo
```

En cada llamada se proporciona un valor de tiempo y la función devuelve un nuevo resultado. No existe una variable que vaya cambiando, el estado se calcula nuevamente a partir de los datos de entrada. Esto es posible por las estructuras de datos de Clojure, cuando parece que un dato se modifica, se crea una nueva versión que comparte gran parte de la estructura de la anterior mediante un structural sharing. De esta forma se conserva la eficiencia sin perder las ventajas de la inmutabilidad.

Este modelo aporta además una gran ventaja en sistemas concurrentes, como los datos no cambian, varios procesos pueden consultar el estado del semáforo simultáneamente sin riesgo de condiciones inconsistentes. Si dos módulos consultan el estado para el mismo instante de tiempo, ambos obtendrán exactamente el mismo resultado.

## **Implementación en Clojure: Requerimiento 1 - Transición.**

### **Especificaciones técnicas:**

**Entrada:** color-actual (símbolo: en-rojo, en-verde, en-amarillo y sus variantes intermitentes) y cambiar-a (símbolo del color destino)

**Salida:** Lista con el estado actual y la acción a realizar

**Comportamiento:** Si la combinación no corresponde a una transición válida del ciclo, retorna accion-por-defecto.

```

(defn transicion [color-actual cambiar-a]
  (cond
    ((and (= color-actual 'en-rojo) (= cambiar-a 'verde-intermitente))
     (list color-actual 'Cambiar-a-verde-intermitente)
    )
    ((and (= color-actual 'en-verde-intermitente) (= cambiar-a 'verde))
     (list color-actual 'Cambiar-a-verde)
    )
    ((and (= color-actual 'en-verde) (= cambiar-a 'amarillo-intermitente))
     (list color-actual 'Cambiar-a-amarillo-intermitente)
    )
    ((and (= color-actual 'en-amarillo-intermitente) (= cambiar-a 'amarillo))
     (list color-actual 'Cambiar-a-amarillo)
    )
    ((and (= color-actual 'en-amarillo) (= cambiar-a 'rojo-intermitente))
     (list color-actual 'Cambiar-a-rojo-intermitente)
    )
    ((and (= color-actual 'en-rojo-intermitente) (= cambiar-a 'rojo))
     (list color-actual 'Cambiar-a-rojo)
    )
    :else
    (list color-actual 'Accion-por-defecto)
  )
)

```

El objetivo de esta función es el mismo que en la implementación en Common Lisp, realizar las transiciones válidas entre los estados del semáforo, en Clojure.

La función recibe el color actual y el color destino, y devuelve una lista con el estado y la acción correspondiente. Si la transición no es válida, retorna el color actual junto con Acción-por-defecto.

En lugar del cond de Common Lisp, se utiliza el cond de Clojure, cuya diferencia sintáctica más notable es que las condiciones no requieren paréntesis extra alrededor de cada rama, y la cláusula por defecto se expresa con la keyword :else. La comparación se realiza con = sobre símbolos cotizados, de manera comparativa en Lisp.

## **Requerimiento 2: Timer.**

**Especificaciones técnicas:**

**Entrada:** Tiempo Unix en número entero.

**Salida:** Símbolo del color activo en ese momento.

**Comportamiento:** Calcula la posición dentro del ciclo y retorna el estado. Un valor negativo produce un resultado inválido por el mod con negativos.

```
(defn timer [unix-time]
  (let [duracion-ciclo 225
        posicion      (mod unix-time duracion-ciclo)]
    (cond
      (< posicion 90)
      'rojo

      (< posicion 93)
      'verde-intermitente

      (< posicion 213)
      'verde

      (< posicion 216)
      'amarillo-intermitente

      (< posicion 222)
      'amarillo

      :else
      'rojo-intermitente)))
```

Esta función cumple el mismo propósito que su equivalente en Common Lisp: recibir un tiempo en formato Unix y determinar qué color del semáforo debe estar activo en ese instante.

Se utiliza let para calcular dos valores que se reutilizan en el cond, la duración total del ciclo y la posición actual dentro de él mediante mod. Esto evita recalcular el módulo tres veces y hace el código más claro. El mod opera igual que en Common Lisp, devuelve el resto de la división, indicando en qué segundo del ciclo actual se encuentra, ignorando todos los ciclos anteriores completados.



## **Conclusión de Clojure:**

La experiencia con Clojure resultó entendible y, a la vez, desafiante. Al pertenecer a la familia Lisp, la transición desde Common Lisp fue más llevadera de lo esperado, pero sus diferencias conceptuales, especialmente el uso de keywords en lugar de símbolos cotizados y la inmutabilidad, obligaron al grupo a repensar cómo modelar el estado y el paso del tiempo sin recurrir a variables mutables. Comprender que el estado no se cambia sino que se calcula a partir de los parámetros de entrada fue el cambio de perspectiva más significativo. En definitiva, Clojure demostró ser un lenguaje moderno que lleva las ideas del paradigma funcional a entornos de producción reales, y su estudio aportó una visión más amplia sobre cómo distintos lenguajes resuelven los mismos problemas con enfoques diferentes.

## **CONCLUSIÓN FINAL:**

El desarrollo de este Trabajo Práctico Integrador representó un desafío significativo para el equipo, tanto desde el punto de vista técnico como organizativo. La necesidad de implementar soluciones en Clojure, implicó un proceso de aprendizaje continuo a lo largo de todas las fases del proyecto.

Durante la Fase 1, la principal dificultad es interiorizar los principios del paradigma funcional, la inmutabilidad, la ausencia de bucles imperativos y el uso de recursividad como mecanismo central de iteración. La resolución de los requerimientos del sistema de semáforos permitió consolidar conceptos como las funciones puras, la composición funcional y el manejo de funciones de Lisp. Los errores documentados en la bitácora de depuración reflejan este proceso, desde errores conceptuales en la lógica de ciclos semafóricos hasta problemas de diseño en la separación entre funciones.

La Fase 2 introdujo al equipo en el ecosistema externo de Common Lisp mediante Quicklisp, reforzando la capacidad de investigar e integrar herramientas de forma autónoma. La incorporación de la librería local-time permitió ampliar las capacidades del sistema de auditoría, añadiendo un formato de salida más legible.

Finalmente, la Fase 3 aportó una perspectiva comparativa a través del estudio de Clojure. El análisis de sus características, en particular el uso de keywords, la inmutabilidad por defecto y las estructuras de datos persistentes, evidenció que, si bien comparte con Common Lisp por pertenecer a la familia Lisp, Clojure presenta soluciones a problemas de concurrencia y estado que resultan especialmente relevantes en el contexto de sistemas y aplicaciones de producción.

En términos generales, el trabajo permitió al grupo adquirir una visión más amplia de la programación funcional, comprendiendo que los principios que rigen este paradigma y consntituyen una forma de pensar los problemas de manera predecible y mantenible.



## Bibliografía:

- Monzón, R. (s.f). Características de LISP.*Paradigmas y Lenguajes: Introducción* [Apuntes de cátedra, archivo PDF]. Facultad de Ciencias Exactas y Naturales y Agrimensura, Universidad Nacional del Nordeste.  
[https://virtual-moodle.unne.edu.ar/pluginfile.php/2328947/mod\\_resource/content/12/PyL%20-%202026%20-%20Introducci%C3%B3n.pdf](https://virtual-moodle.unne.edu.ar/pluginfile.php/2328947/mod_resource/content/12/PyL%20-%202026%20-%20Introducci%C3%B3n.pdf)
- Monzón, R. (s.f). Características de LISP.*Paradigmas y Lenguajes: Funcional Parte I* [Apuntes de cátedra, archivo PDF]. Facultad de Ciencias Exactas y Naturales y Agrimensura, Universidad Nacional del Nordeste.  
[https://virtual-moodle.unne.edu.ar/pluginfile.php/2328949/mod\\_resource/content/13/PyL%20-%202026%20-%20Funcional%20Parte%201.pdf](https://virtual-moodle.unne.edu.ar/pluginfile.php/2328949/mod_resource/content/13/PyL%20-%202026%20-%20Funcional%20Parte%201.pdf)
- Wikipedia contributors. (s.f.). Tiempo Unix. En Wikipedia, La enciclopedia libre. URL:  
[https://es.wikipedia.org/wiki/Tiempo\\_Unix](https://es.wikipedia.org/wiki/Tiempo_Unix)
- Hickey, R. (2012). Are we there yet? InfoQ. URL:  
<https://www.infoq.com/presentations/Are-We-There-Yet-Rich-Hickey/>
- Halloway, S., & Bedra, A. (2012). Programming Clojure (2nd ed.). Pragmatic Bookshelf.
- Fogus, M., & Houser, C. (2015). The Joy of Clojure (2nd ed.). Manning Publications.
- Wikipedia contributors. (2026). Clojure. Wikipedia. URL:  
<https://en.wikipedia.org/wiki/Clojure>
- Hickey, R. (2009). The value of values. InfoQ. URL:  
<https://www.infoq.com/presentations/Value-Values/>
- Clojure. (s.f.). Data structures. Clojure.org. URL:  
[https://clojure.org/reference/data\\_structures](https://clojure.org/reference/data_structures)
- Clojure. (s.f.). Keywords. Clojure.org. URL:  
[https://clojure.org/reference/reader#\\_literals](https://clojure.org/reference/reader#_literals)
- Beane, Z. (s. f.). Quicklisp beta. Quicklisp. URL:  
<https://www.quicklisp.org/beta/>
- Argentina. Ministerio de Justicia y Derechos Humanos. (s. f.). *Texto actualizado de la norma*. InfoLEG. URL:  
<https://servicios.infoleg.gob.ar/infolegInternet/anexos/0-4999/818/texact.htm>